



Topics in Software Engineering – Software Reengineering

By

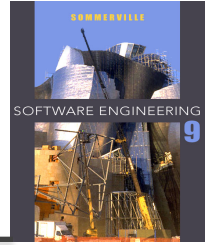
Dr. Junaid Akram

Assistant Professor, Department of Computer Science COMSATS (Lahore)



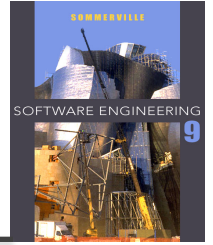
Software Reengineering & Legacy Systems

System Re-Engineering



- ✧ Older legacy systems, are difficult to understand and change.
- ✧ Re-structuring or re-writing part or all of a legacy system without changing its functionality
- ✧ Applicable where some but not all sub-systems of a larger system require frequent maintenance
- ✧ Re-engineering involves adding effort to make them easier to maintain. The system may be re-structured and re-documented.

Advantages of Re-engineering



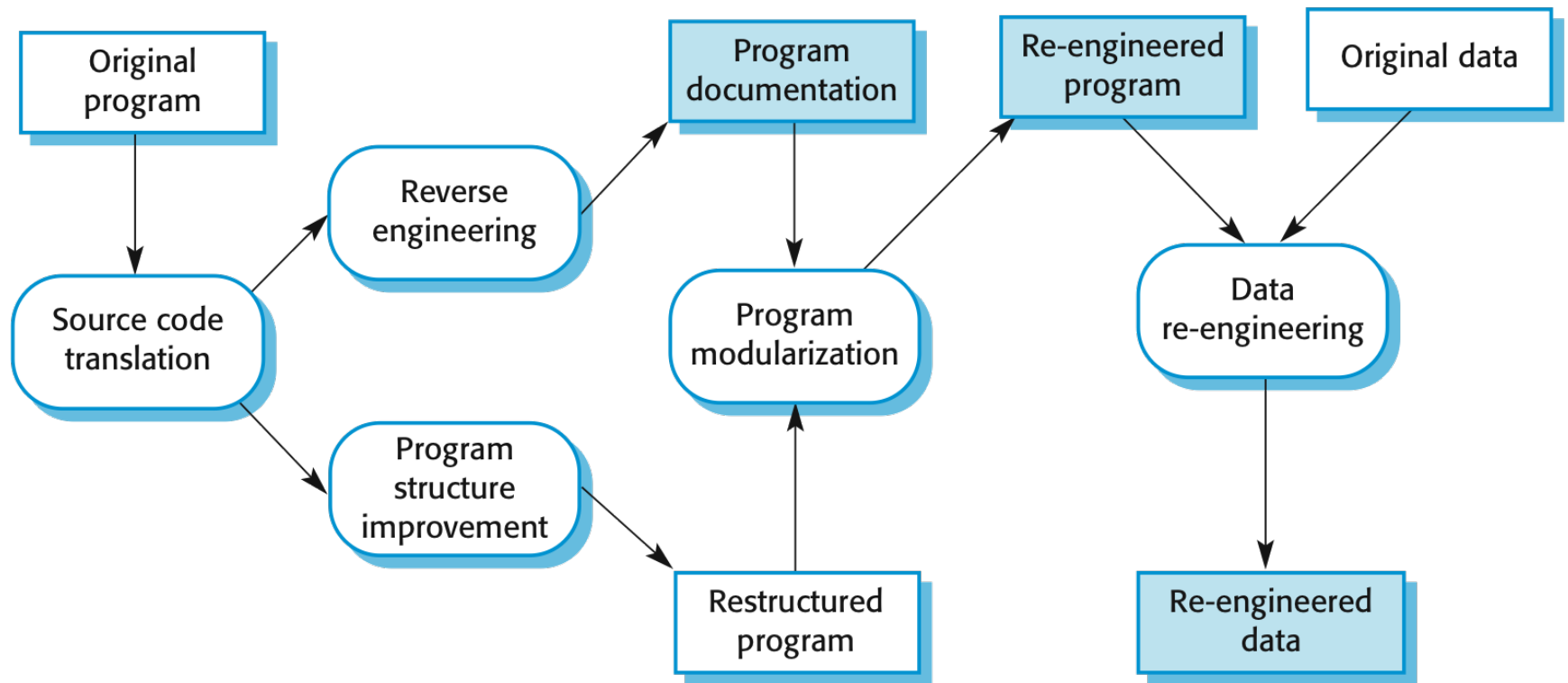
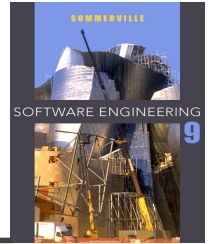
Reduced risk

- There is a high risk in new software development. There may be development problems, staffing problems and specification problems.

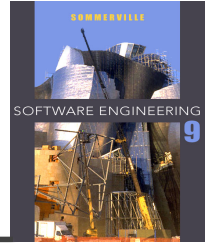
Reduced cost

- The cost of re-engineering is often significantly less than the costs of developing new software.

The Reengineering Process

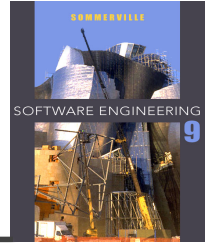


Reengineering Process Activities



- ✧ The input to the process is a legacy program and the output is an improved and restructured version of the same program.
- ✧ Source code translation
 - Using a translation tool, the program is converted from an old programming language to a more modern version of the same language or to a different
- ✧ Reverse engineering
 - The program is analyzed and information extracted from it. This helps to document its organization and functionality. Again, this process is usually completely automated.

Cont..



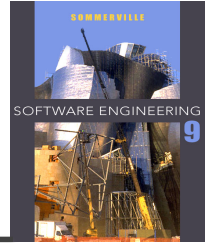
✧ Program structure improvement

- The control structure of the program is analyzed and modified to make it easier to read and understand. This can be partially automated, but some manual intervention is usually required.

✧ Program modularization

- Related parts of the program are grouped together and, where appropriate, redundancy is removed.
- In some cases, this stage may involve architectural refactoring (e.g., a system that uses several different data stores may be refactored to use a single repository). This is a manual process.

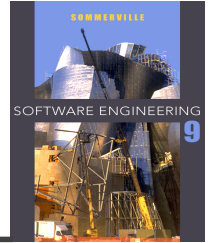
Cont..



✧ Data reengineering

- The data processed by the program is changed to reflect program changes. This may mean redefining database schemas and converting existing databases to the new structure. You should usually also clean up the data. This involves finding and correcting mistakes, removing duplicate records etc. Tools are available to support data reengineering.

Reengineering Cost Factors



The quality of the software to be reengineered



The tool support available for reengineering



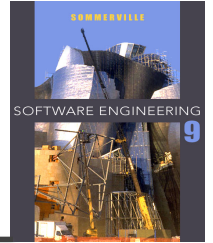
The extent of the data conversion which is required



The availability of expert staff for reengineering

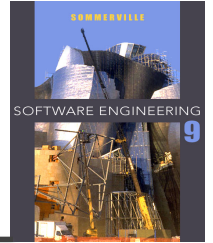
This can be a problem with old systems based on technology that is no longer widely used

Preventive Maintenance by Refactoring



- ✧ **Refactoring** is the process of making improvements to a program to slow down degradation through change
- ✧ You can think of refactoring as ‘**preventive maintenance**’ that reduces the problems of future change
- ✧ Refactoring involves modifying a program to improve its structure, reduce its complexity or make it easier to understand
- ✧ When you refactor a program, **you should not add functionality but rather concentrate on program improvement**

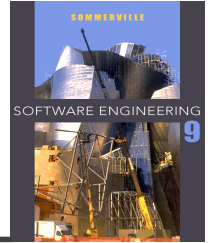
Refactoring vs. Reengineering



Re-engineering takes place after a system has been maintained for some time and maintenance costs are increasing. You use automated tools to process and re-engineer a legacy system to create a new system that is more maintainable.

Refactoring is a continuous process of improvement throughout the development and evolution process. It is intended to avoid the structure and code degradation that increases the costs and difficulties of maintaining a system.

Legacy System Management



Organizations that rely on legacy systems must choose a strategy for evolving these systems

- **Scrap the system completely**

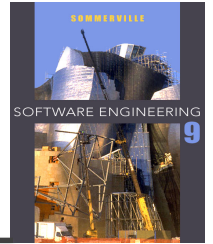
This option should be chosen when the system is not making an effective contribution to business processes.

This commonly occurs when business processes have changed since the system was installed and are no longer reliant on the legacy system.

- **Continue maintaining the system**

This option should be chosen when the system is still required but is fairly stable and the system users make relatively few change requests.

Cont..



- **Transform the system** by re-engineering to improve its **maintainability**; This option should be chosen when the system quality has been degraded by change and where a new change to the system is still being proposed. This process may include developing new interface components so that the original system can work with other, newer systems.
- **Replace the system with a new system.** This option should be chosen when factors, such as new hardware, mean that the old system cannot continue in operation or where off-the-shelf systems would allow the new system to be developed at a reasonable cost. In many cases, an evolutionary replacement strategy can be adopted in which major system components are replaced by off-the-shelf systems with other components reused wherever possible.

The strategy chosen should depend on the system quality and its business value.

Legacy System Categories

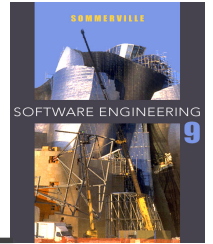
Low quality, low business value

- Keeping these systems in operation will be expensive and the rate of the return to the business will be fairly small. These systems should be scrapped.

Low-quality, high-business value

- These systems are making an important business contribution so they cannot be scrapped. However, their low quality means that it is expensive to maintain them.
- These systems should be reengineered to improve their quality.
- They may be replaced, if a suitable off-the-shelf system is available.

Cont..



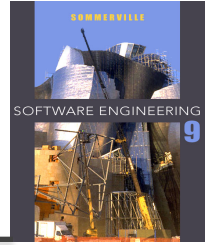
High-quality, low-business value

- These are systems that don't contribute much to the business but which may not be very expensive to maintain. It is not worth replacing these systems so normal system maintenance may be continued if expensive changes are not required and the system hardware remains in use.
- If expensive changes become necessary, the software should be scrapped. with COTS, scrap completely or maintain

High-quality, high business value

- These systems have to be kept in operation.
- However, their high quality means that you don't have to invest in transformation or system replacement. Normal system maintenance should be continued. in operation using normal system maintenance

Business Value Assessment



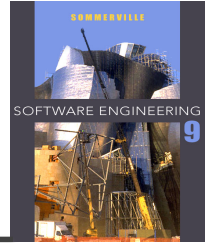
Assessment should take different viewpoints into account

- System end-users
- Business customers
- Line managers
- IT managers
- Senior managers



Interview different stakeholders and collate results

Issues in Business Value Assessment



The use of the system

If systems are only used occasionally or by a small number of people, they may have a low business value



The business processes that are supported

A system may have a low business value if it forces the use of inefficient business processes



System dependability

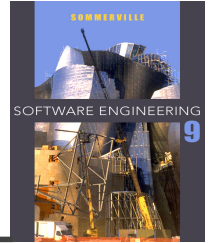
If a system is not dependable and the problems directly affect business customers, the system has a low business value



The system outputs

If the business depends on system outputs, then the system has a high business value

System Quality Assessment



Business process assessment

- How well does the business process support the current goals of the business?

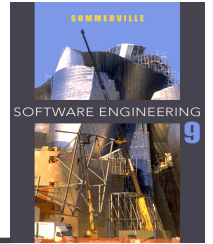
Environment assessment

- How effective is the system's environment and how expensive is it to maintain?

Application assessment

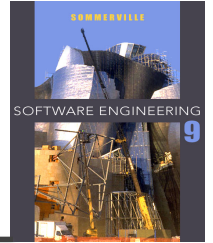
- What is the quality of the application software system?

Example



- ✧ A company provides a travel ordering system that is used by staff responsible for arranging travel. They can place orders with an approved travel agent. Tickets are then delivered and the company is invoiced for these.
- ✧ However, a business value assessment may reveal that this system is only used for a fairly small percentage of travel orders placed. People making travel arrangements find it cheaper and more convenient to deal directly with travel suppliers through their websites. This system may still be used, but there is no real point in keeping it. The same functionality is available from external systems.

Another Example

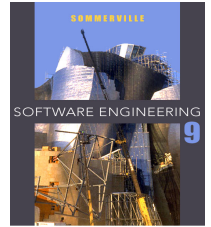


- ✧ A company has developed a system that keeps track of all previous customer orders and automatically generates reminders for customers to reorder goods.
- ✧ This results in a large number of repeat orders and keeps customers satisfied because they feel that their supplier is aware of their needs. The outputs from such a system are very important to the business and this system therefore has a high business value.

Factors used in Environment Assessment

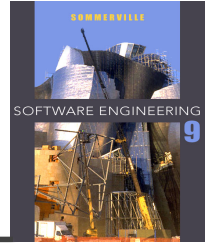
Factor	Questions
Supplier stability	Is the supplier still in existence? Is the supplier financially stable and likely to continue in existence? If the supplier is no longer in business, does someone else maintain the systems?
Failure rate	Does the hardware have a high rate of reported failures? Does the support software crash and force system restarts?
Age	How old is the hardware and software? The older the hardware and support software, the more obsolete it will be. It may still function correctly but there could be significant economic and business benefits to moving to a more modern system.
Performance	Is the performance of the system adequate? Do performance problems have a significant effect on system users?
Support requirements	What local support is required by the hardware and software? If there are high costs associated with this support, it may be worth considering system replacement.
Maintenance costs	What are the costs of hardware maintenance and support software licences? Older hardware may have higher maintenance costs than modern systems. Support software may have high annual licensing costs.
Interoperability	Are there problems interfacing the system to other systems? Can compilers, for example, be used with current versions of the operating system? Is hardware emulation required?

Factors used in Application Assessment



Factor	Questions
Understandability	How difficult is it to understand the source code of the current system? How complex are the control structures that are used? Do variables have meaningful names that reflect their function?
Documentation	What system documentation is available? Is the documentation complete, consistent, and current?
Data	Is there an explicit data model for the system? To what extent is data duplicated across files? Is the data used by the system up to date and consistent?
Performance	Is the performance of the application adequate? Do performance problems have a significant effect on system users?
Programming language	Are modern compilers available for the programming language used to develop the system? Is the programming language still used for new system development?
Configuration management	Are all versions of all parts of the system managed by a configuration management system? Is there an explicit description of the versions of components that are used in the current system?
Test data	Does test data for the system exist? Is there a record of regression tests carried out when new features have been added to the system?
Personnel skills	Are there people available who have the skills to maintain the application? Are there people available who have experience with the system?

System Measurement



✧ You may collect **quantitative data** to make an assessment of the quality of the application system

- The number of system change requests:

System changes usually corrupt the system structure and make further changes more difficult. The higher this accumulated value, the lower the quality of the system.

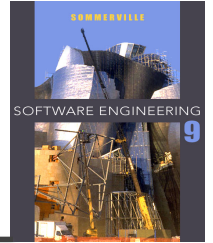
- The number of different user interfaces used by the system:

This is an important factor in forms-based systems where each form can be considered as a separate user interface. The more interfaces, the more likely that there will be inconsistencies and redundancies in these interfaces.

- The volume of data used by the system:

The higher the volume of data (number of files, size of database, etc.), the more likely that it is that there will be data inconsistencies that reduce the system quality.

Key points



Software development and evolution can be thought of as an integrated, iterative process that can be represented using a spiral model

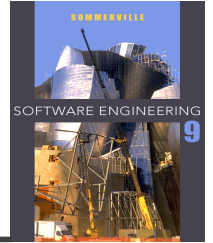


For custom systems, the costs of software maintenance usually exceed the software development costs



The process of software evolution is driven by requests for changes and includes change impact analysis, release planning and change implementation

Key points



There are 3 types of software maintenance, namely bug fixing, modifying software to work in a new environment, and implementing new or changed requirements



Software re-engineering is concerned with re-structuring and re-documenting software to make it easier to understand and change

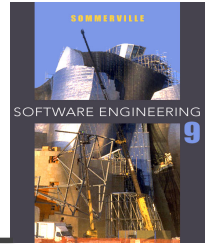


Refactoring, making program changes that preserve functionality, is a form of preventative maintenance.



The business value of a legacy system and the quality of the application should be assessed to help decide if a system should be replaced, transformed or maintained

Thanks for your attention!



Any Question?



Email me on : junaidakram@cuilahore.edu.pk